CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Subject: 24CSH-337

Subject code: Computer Networks

Assignment No.: 3

Student Name: Husanpreet Kaur

UID: 24BCY70138

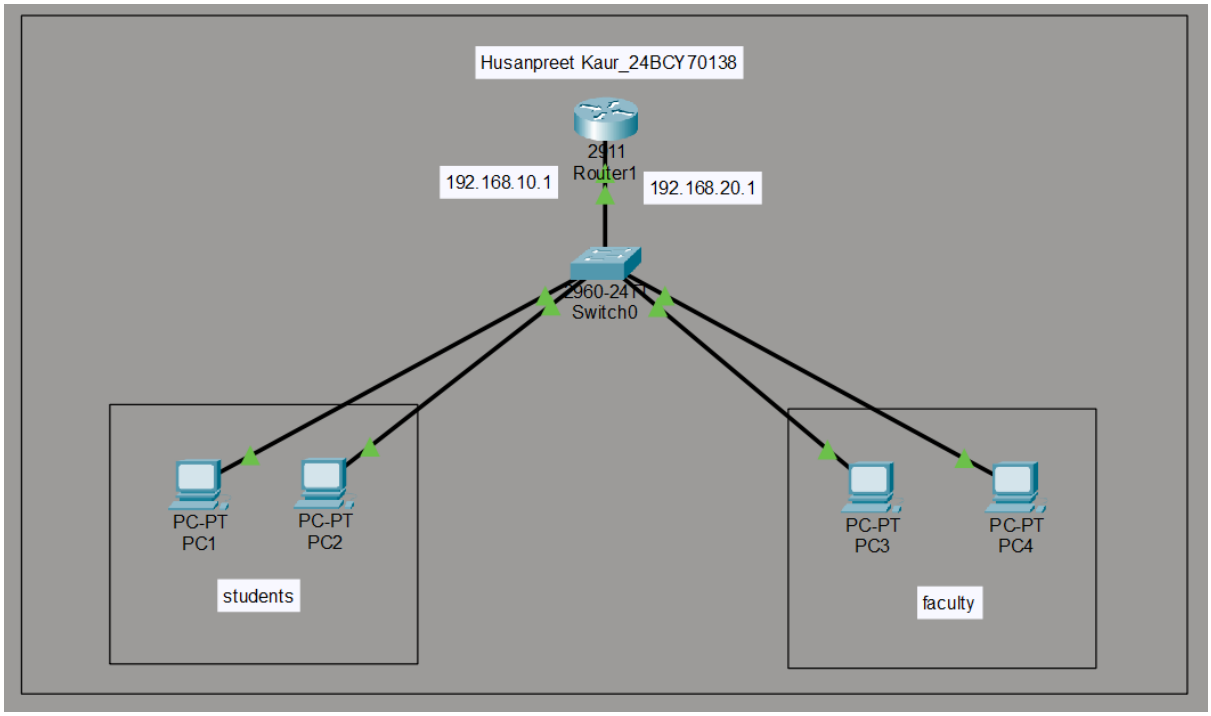
Branch: AIT-CSE (Cyber Security)

Submitted to: Mr. Alok Kumar

Date of Submission: 21/10/2026

Q1: Implement router as DHCP server for serving multiple VLANs.

Sol:



Step 1: Plan the IP addressing:

Group	VLAN ID	Network	Gateway
Students (PC1, PC2)	10	192.168.10.0/24	192.168.10.1
Faculty (PC3, PC4)	20	192.168.20.0/24	192.168.20.1

Step 2: Create VLANs on Switch0

Click Switch0 → CLI and enter:

enable

configure terminal

vlan 10

name STUDENTS

exit

vlan 20

name FACULTY

exit

Step 3: Assign switch ports to VLANs

Assuming the router is connected to Fa0/1, PC1 to Fa0/2, PC2 to Fa0/3, PC3 to Fa0/4, and PC4 to Fa0/5, enter:

interface range fa0/2-3

switchport mode access

switchport access vlan 10

exit

interface range fa0/4-5

switchport mode access

switchport access vlan 20

exit

Configure the router-facing port as a trunk:

interface fa0/1

switchport mode trunk

switchport trunk allowed vlan 10,20

no shutdown

exit

end

write memory

Step 4: Configure router sub-interfaces

Click Router1 → CLI and enter:

enable

configure terminal

interface gigabitEthernet 0/0

no shutdown

exit

interface gigabitEthernet 0/0.10

encapsulation dot1Q 10

ip address 192.168.10.1 255.255.255.0

exit

interface gigabitEthernet 0/0.20

encapsulation dot1Q 20

ip address 192.168.20.1 255.255.255.0

exit

Step 5: Configure the router as the DHCP server

First, exclude addresses reserved for gateways and network devices:

ip dhcp excluded-address 192.168.10.1 192.168.10.10

ip dhcp excluded-address 192.168.20.1 192.168.20.10

Create the Students DHCP pool:

ip dhcp pool STUDENTS

network 192.168.10.0 255.255.255.0

default-router 192.168.10.1

dns-server 8.8.8.8

exit

Create the Faculty DHCP pool:

ip dhcp pool FACULTY

network 192.168.20.0 255.255.255.0

default-router 192.168.20.1

dns-server 8.8.8.8

exit

end

write memory

Step 6: Enable DHCP on all four PCs

For each PC:

1. Click the PC.
2. Select Desktop.
3. Open IP Configuration.
4. Click DHCP.
5. Wait for the IP address to appear.

Expected addressing:

PC	VLAN	Example assigned IP
PC1	10	192.168.10.11
PC2	10	192.168.10.12
PC3	20	192.168.20.11
PC4	20	192.168.20.12

The actual addresses may differ depending on the order of DHCP requests.

Step 7: Verify the configuration

On the switch:

show vlan brief

show interfaces trunk

On the router:

show ip interface brief

show ip dhcp pool

show ip dhcp binding

Confirm that both router sub-interfaces are up and that the DHCP bindings list the PCs that received addresses.

Step 8: Test connectivity

On PC1, open Desktop → Command Prompt and run:

ping 192.168.10.1

Test communication with another student PC:

ping 192.168.10.12

Test communication with a faculty PC:

ping 192.168.20.11

If these tests succeed, DHCP and inter-VLAN routing are working.

Q2: Explain 3-way handshake for TCP.

Sol: The TCP (Transmission Control Protocol) 3-way handshake is a process used to establish a reliable connection between a client and a server before they begin exchanging data.

TCP is a connection-oriented transport layer protocol in the TCP/IP model. Before transmitting application data, TCP establishes a connection and synchronizes sequence numbers between the two communicating devices.

The three steps of the handshake are:

1. SYN — Synchronize
2. SYN-ACK — Synchronize + Acknowledge
3. ACK — Acknowledge

The handshake ensures that both the client and the server are ready to communicate and can send and receive TCP segments.

Step 1: SYN — Client requests a connection

The client sends a TCP segment to the server with the SYN flag set to 1.

SYN stands for Synchronize.

This message tells the server that the client wants to establish a TCP connection and provides the client's initial sequence number.

For example:

- Client's initial sequence number (ISN): 100
- SYN flag: 1
- Destination port: 80 for HTTP, or 443 for HTTPS
- Connection state: Client enters SYN-SENT

The client is effectively saying:

"Hello, server! I want to establish a TCP connection. My initial sequence number is 100."

The server receives the SYN segment and prepares to respond.

Step 2: SYN-ACK — Server acknowledges the request

If the server is listening and accepts the connection request, it responds with a TCP segment containing both the SYN and ACK flags set to 1.

This response performs two functions:

1. ACK: Acknowledges the client's SYN.
2. SYN: Sends the server's own initial sequence number.

For example:

- Server's initial sequence number: 500
- SYN flag: 1
- ACK flag: 1
- Acknowledgment number: 101
- Connection state: Server enters SYN-RECEIVED

Why is the acknowledgment number 101?

Because TCP acknowledges the next sequence number it expects. A SYN consumes one sequence number, so:

Acknowledgment number = 100 + 1 = 101

The server is effectively saying:

"I received your request and acknowledge sequence number 100. My initial sequence number is 500."

At this point, the server has acknowledged the client's connection request, but the client still needs to acknowledge the server's SYN.

Step 3: ACK — Client confirms the connection

The client receives the SYN-ACK and sends a final TCP segment with the ACK flag set to 1.

For example:

- ACK flag: 1
- Acknowledgment number: 501
- Sequence number: 101
- Connection state: Client enters ESTABLISHED

Why is the acknowledgment number 501?

The server's SYN consumes one sequence number:

Acknowledgment number = 500 + 1 = 501

The client is effectively saying:

"I received your response and acknowledge your initial sequence number. We can now communicate."

When the server receives this final ACK, its connection state becomes ESTABLISHED as well.

The three-way handshake is now complete, and both sides can exchange application data.

TCP concepts used:

A. Sequence number: A sequence number identifies the position of bytes in the TCP data stream. It helps TCP deliver data in order and detect missing data.

During connection establishment, each side chooses an initial sequence number (ISN).

B. Acknowledgment number: The acknowledgment number indicates the next sequence number expected from the other endpoint.

For example, if a client sends a SYN with sequence number 100, the server acknowledges it with ACK number 101.

C. TCP flags:

Flag	Meaning	Purpose
SYN	Synchronize	Establishes a connection and synchronizes sequence numbers
ACK	Acknowledgment	Acknowledges received sequence numbers
FIN	Finish	Initiates an orderly connection shutdown
RST	Reset	Resets or aborts a connection
PSH	Push	Requests prompt delivery of received data to the application
URG	Urgent	Indicates urgent data through the urgent pointer mechanism

D. Port numbers: Port numbers identify the applications or services involved in a connection.

For example, a browser may use a temporary client-side port to connect to a web server on port 443 for HTTPS.

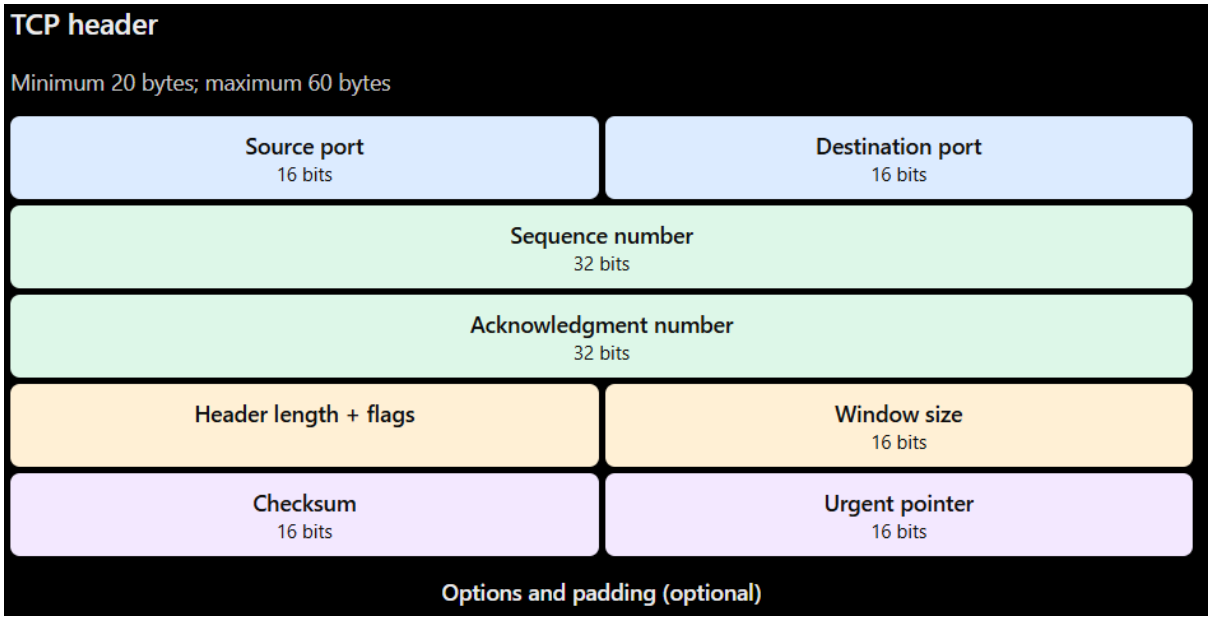
The connection is identified by the combination of source IP, source port, destination IP, and destination port, known as the four-tuple.

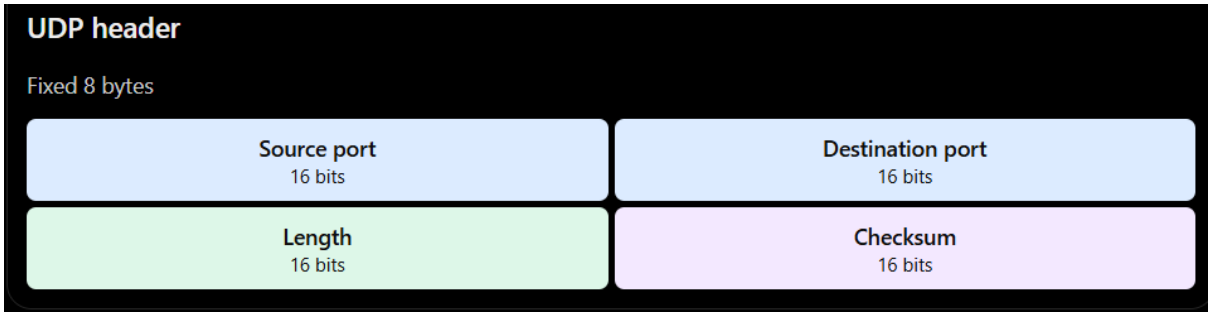
Q3: What is the difference between TCP and UDP headers? Elaborate in detail.

Sol: TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are transport-layer protocols in the TCP/IP model. Both are responsible for delivering data between applications running on different devices, but they use different header structures because they provide different communication services.

- TCP provides reliable, connection-oriented communication with acknowledgments, sequencing, retransmissions and flow control.
- UDP provides connectionless communication with a smaller header and minimal protocol overhead.

The header is the control information added before the application data. It contains fields that help the receiving device process and deliver the data correctly.





The main difference is that TCP has additional fields for reliable data transfer and connection management, while UDP contains only the fields needed for basic datagram delivery.

Detailed explanation of TCP header fields:

The TCP header has a minimum size of 20 bytes and a maximum size of 60 bytes, depending on whether optional fields are present.

1. Source port (16 bits)

Identifies the application or process sending the data.

For example, a client might use source port 51500 when connecting to a web server.

2. Destination port (16 bits)

Identifies the destination application or service.

Examples include:

- Port 80 — HTTP
- Port 443 — HTTPS
- Port 25 — SMTP

Together, the source and destination ports help TCP deliver data to the correct application endpoint.

3. Sequence number (32 bits)

Identifies the position of data bytes in the TCP stream.

It enables TCP to:

- Arrange data in the correct order.
- Identify missing data.
- Support retransmission of lost data.

The SYN flag also consumes one sequence number during connection establishment.

4. Acknowledgment number (32 bits)

Indicates the next sequence number expected from the other endpoint when the ACK flag is set.

For example, if a receiver has received bytes up to sequence number 199, it can acknowledge with 200, indicating that byte 200 is expected next.

5. Data offset / header length (4 bits)

Specifies the size of the TCP header in 32-bit words.

- Minimum value: 5 words = 20 bytes.
- Maximum value: 15 words = 60 bytes.

This field helps the receiver determine where the application data begins.

6. Control flags

TCP uses control flags to manage connections and data transfer.

Flag	Purpose
SYN	Initiates a connection and synchronizes sequence numbers
ACK	Indicates that the acknowledgment field is valid
FIN	Initiates an orderly connection shutdown

RST	Resets a connection
PSH	Requests prompt delivery of received data to the application
URG	Indicates that the urgent pointer is meaningful
ECE, CWR	Support Explicit Congestion Notification (ECN)
NS	An additional ECN-related flag defined in the TCP header

The six traditional flags are SYN, ACK, FIN, RST, PSH and URG. Modern TCP headers also include ECN-related control bits.

7. Window size (16 bits)

The window size is used for flow control.

It tells the sender how much additional data the receiver is currently willing to accept, helping prevent the sender from overwhelming the receiver's buffer.

TCP window scaling, when negotiated, allows larger effective window sizes than the basic 16-bit field alone.

8. Checksum (16 bits)

Checks for corruption in the TCP segment, including the TCP header and data. The checksum calculation also incorporates an IP pseudo-header.

If corruption is detected, TCP can rely on retransmission mechanisms to recover missing or damaged data.

9. Urgent pointer (16 bits)

When the URG flag is set, this field helps identify the end of the urgent data region according to TCP's urgent-data mechanism.

It is rarely used in modern applications.

10. Options and padding (optional)

TCP options provide additional capabilities, such as:

- MSS (Maximum Segment Size): Negotiates the maximum TCP data size in a segment.
- Window scaling: Supports larger receive windows.
- SACK (Selective Acknowledgment): Helps receivers report which data blocks have arrived.

- Timestamps: Support functions such as round-trip-time measurement and protection against certain old duplicate segments.

Padding ensures the TCP header ends on a 32-bit boundary.

Detailed explanation of UDP header fields:

It is always 8 bytes (64 bits) long and contains four fields.

1. Source port (16 bits)

Identifies the sending application's port. It can be zero when the sender does not need a reply to be directed to a specific port.

2. Destination port (16 bits)

Identifies the destination application or service.

For example, a DNS query commonly uses destination port 53.

3. Length (16 bits)

Specifies the total length of the UDP header and UDP data, in bytes.

- Minimum length: 8 bytes.
- Maximum representable length: 65,535 bytes, although practical UDP datagram sizes are constrained by the IP version and network MTU.

4. Checksum (16 bits)

Provides error detection for the UDP header and data, using an IP pseudo-header as part of the calculation.

- In IPv4, a UDP checksum of zero indicates that the sender did not calculate a checksum.
- In IPv6, UDP checksums are generally mandatory, with narrowly defined exceptions.

Important: UDP checksums detect corruption; they do not provide retransmission, delivery guarantees or correction of damaged data.

Feature	TCP header	UDP header
Header size	20–60 bytes	Exactly 8 bytes
Source port	16 bits	16 bits
Destination port	16 bits	16 bits

Sequence number	Present, 32 bits	Absent
Acknowledgment number	Present, 32 bits	Absent
Header length field	Present	Absent; fixed header size
Control flags	Multiple control flags	No TCP-style control flags
Window size	Present for flow control	Absent
Length field	No direct total-segment-length field in the TCP header	Present; header + data length
Checksum	16 bits	16 bits
Options	Optional	None in the UDP header
Connection management	Supported	Not provided by UDP
Reliability mechanisms	Acknowledgments and retransmissions	Not provided by UDP itself
Ordering	TCP stream preserves byte order	No ordering guarantee
Flow control	Built in	Not provided by UDP itself
Typical overhead	Higher	Lower